



Προγραμματισμός Συστημάτων

2025 - 2026

1η Εργαστηριακή Άσκηση

Ομάδα Project

Οι ομάδες project θα πρέπει να αποτελούνται από 1 έως 2 άτομα.

Παραδοτέα Project

Τα παραδοτέα κάθε ομάδας είναι:

- Η αναφορά του project της ομάδας, σε μορφή pdf.
- Τα αρχεία του κώδικα της υλοποίησης σε συμπιεσμένη μορφή zip. **Στην αρχή του αρχείου του κώδικα, σε σχόλια, θα τοποθετείτε τα ονοματεπώνυμα και τους ΑΜ των μελών της ομάδας ως εξής:**

```
// Panagiotis Hadijdoukas, 3456789
```

```
// Giorgos Papadimitrou Ilias, 1234567
```

Τα παραδοτέα θα υποβληθούν μέσω του eclass στις «Εργασίες», όπου θα παραδώσει ένα μέλος της ομάδας και θα είναι το ίδιο μέλος για όλα τα παραδοτέα (και για τις επόμενες δύο ασκήσεις).

Θα παραδοθούν:

- η αναφορά σε pdf στην εργασία «1η εργασία 2025-2026, Αναφορά Ομάδων»
- τα αρχεία του κώδικα σας σε μορφή zip στην εργασία «1η εργασία 2025-2026, Κώδικας Ομάδων»

Στο περιεχόμενο της αναφοράς της κάθε ομάδας, η τεκμηρίωση είναι απαραίτητη και θα αξιολογηθεί. Στο εξώφυλλο της αναφοράς, πρέπει να περιλαμβάνει τα ονόματα των μελών της ομάδας, τους αριθμούς μητρώου και το email τους.

Θα αποτελείται από μια σύντομη περιγραφή του σχεδιασμού της υλοποίησης κάθε ομάδας. Επίσης, θα περιγράφει εν συντομία τα προβλήματα που αντιμετωπίσατε κατά την υλοποίηση της άσκησης και τις προσεγγίσεις της ομάδας για την επίλυση τους. Εάν εντοπίσατε περισσότερες από μια προσεγγίσεις για την επίλυση, να τις αναφέρετε και να τεκμηριώσετε γιατί επιλέξατε όποια επιλέξατε.

Ειδικά και επιπρόσθετα, στις ασκήσεις που πρέπει να αναπτύξετε κάποιας μορφής κώδικα/πρόγραμμα, ο κώδικας σας πρέπει να απαντά ακριβώς σε ό,τι σας ζητείτε, χωρίς αφαιρέσεις ή προσθήκες. Οι απαντήσεις που αφορούν κώδικα μέσα στο pdf της αναφοράς θα πρέπει να τεκμηριώνονται με screenshots από την εκτέλεση των προγραμμάτων.

Στην αναφορά (pdf), στο μέρος που αντιστοιχεί στις υλοποιήσεις σας, πρέπει να αναφέρει στην αρχή της τεκμηρίωσης:

- ποια τμήματα της άσκησης υλοποιήσατε και ποια όχι
- ποια δουλεύουν σωστά και ποια δεν δουλεύουν.

ΜΗΝ συμπεριλάβετε στην αναφορά σας την εκφώνηση της άσκησης και τον κώδικά σας αυτούσιο, αλλά καλείστε να περιγράψετε λεκτικά τον αλγόριθμο, κώδικα και προσέγγιση που ακολουθήσατε.



Επικοινωνία και παράδοση του project

Όλη η επικοινωνία του μαθήματος και κατά συνέπεια και όλη η επικοινωνία για το project και η παράδοση του project θα γίνεται αποκλειστικά μέσω του eClass του μαθήματος.

Οι απορίες θα συζητούνται **πρωτίστως στο μάθημα** και δευτερευόντως με μήνυμα («Μηνύματα» του eclass) μέσω του eClass.

Η παράδοση θα γίνει μέσω eClass, μέσα από τις «Εργασίες» του μαθήματος.

Απορίες με e-mail και γενικά κάθε επικοινωνία μέσω e-mail θα αγνοείται.

Η οριστική ημερομηνία παράδοσης του project θα είναι ίδια για όλους χωρίς εξαιρέσεις και θα εμφανισθεί στο eclass (Εργασίες). **Τα παραδοτέα δεν μπορούν να αλλαχθούν μετά την καταληκτική ημερομηνία παράδοσης μέσω του eclass.**

Μετά την παρέλευση της παράδοσης του project, κανένα project δε θα γίνεται δεκτό μέσω e-mail, με οποιαδήποτε δικαιολογία.

Περιπτώσεις αντιγραφής και λογοκλοπής

Projects που είναι προϊόντα αντιγραφής ή λογοκλοπής θα μηδενίζονται και ανάλογα με τη σοβαρότητα της περίπτωσης το θέμα θα παραπέμπεται στη Επιτροπή Δεοντολογίας του Τμήματος.



Εργαστηριακή Άσκηση

Υπολογισμός ολοκληρώματος συνάρτησης

Ο παρακάτω κώδικας υπολογίζει το ολοκλήρωμα της συνάρτησης $f(x) = \sin(\cos(x))$ στο διάστημα $[0, 1]$, χρησιμοποιώντας τη μέθοδο Monte Carlo. Ο κώδικας του ερωτήματος είναι διαθέσιμος στο αντίστοιχο αρχείο zip, στο e-class του μαθήματος. Στο αρχείο είναι διαθέσιμο το Makefile αλλά και 4 κενά αρχεία κώδικα τα οποία θα πρέπει να συμπληρώσετε.

```
// integral_mc_seq.c: Monte Carlo integration - sequential code
// https://en.wikipedia.org/wiki/Monte_Carlo_integration
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <sys/time.h>

double get_wtime(void) {
    struct timeval t;
    gettimeofday(&t, NULL);
    return (double)t.tv_sec + (double)t.tv_usec * 1.0e-6;
}

inline double f(double x) {
    return sin(cos(x));
}

// WolframAlpha: integral sin(cos(x)) from 0 to 1 = 0.738643
// 0.73864299803689018
// 0.7386429980368901838000902905852160417480209422447648518714116299

int main(int argc, char *argv[]) {
    double a = 0.0;
    double b = 1.0;
    unsigned long n = 24e7; // or e8
    const double h = (b-a)/n;
    const double ref = 0.73864299803689018;
    const long tseed = 10;
    double res = 0;
    double t0, t1;

    if (argc == 2) {
        n = atol(argv[1]);
    }

    srand48(tseed);

    t0 = get_wtime();
    for (unsigned long i = 0; i < n; i++) {
        double xi;
        xi = drand48();
        res += f(xi);
    }
    res *= h;
    t1 = get_wtime();
    printf("Result=%.16f Error=%e Rel.Error=%e Time=%lf seconds\n",
        res, fabs(res-ref), fabs(res-ref)/ref, t1-t0);
    return 0;
}
```

Ζητούμενο της άσκησης είναι η υλοποίηση του κώδικα χρησιμοποιώντας πολλαπλές διεργασίες διάφορους μηχανισμούς διαδιεργασιακής επικοινωνίας. Οι διεργασίες πρέπει να διαμοιράζονται τις επαναλήψεις του βρόγχου, να υπολογίζουν το μερικό αποτέλεσμα και να χρησιμοποιούν τον μηχανισμό επικοινωνίας για να ενημερώσουν τη γονική διεργασία. Ο αριθμός των διεργασιών θα πρέπει να ορίζεται **δυναμικά**, π.χ. να δίνεται ως όρισμα στη γραμμή εκτέλεσης ή να διαβάζεται κατά τον χρόνο εκτέλεσης.



Συγκεκριμένα, θα πρέπει να υλοποιηθούν οι παρακάτω περιπτώσεις:

- a) `integral_mc_mq.c`: κάθε διεργασία-εργάτης θα στείλει το αποτέλεσμα της μέσω ουράς μηνυμάτων.
- b) `integral_mc_fifo.c`: κάθε διεργασία-εργάτης θα στείλει το αποτέλεσμα της γράφει το αποτέλεσμα της με μηχανισμό `fifo`.
- c) `integral_mc_mmap.c`: κάθε διεργασία-εργάτης θα γράφει το αποτέλεσμα της σε μία ξεχωριστή θέση ενός πίνακα που έχει δεσμευτεί σε κοινή μνήμη. Στο τέλος, η πατρική διεργασία θα αθροίσει τα μερικά αποτελέσματα για να υπολογίσει το συνολικό.
- d) `integral_mc_mmap_sem.c`: κάθε διεργασία-εργάτης θα γράφει το αποτέλεσμα της σε μία μοναδική θέση μνήμης (π.χ. πίνακα ενός μόνο στοιχείου) που έχει δεσμευτεί σε κοινή μνήμη. Η ανανέωση της κοινής θέσης μνήμης αποτελεί κρίσιμη περιοχή που πρέπει να προστατευτεί με σημαφόρο.

Παρατηρήσεις:

- Η κοινή/διαμοιραζόμενη μνήμη (shared memory) θα πρέπει να δημιουργηθεί με την κλήση συστήματος `mmap`.
- Κάθε διεργασία-εργάτης πρέπει να αρχικοποιήσει τη γεννήτρια τυχαίων αριθμών με διαφορετικό `seed`.
- Αναφέρετε χρόνους εκτέλεσης κάθε λύσης για διαφορετικό αριθμό διεργασιών, προσθέτοντας κατάλληλο πίνακα ή γραφική παράσταση στην αναφορά σας.
- Bonus: προσπαθήστε να χρησιμοποιήσετε και την αρχική γονική διεργασία στους υπολογισμούς σας. Έτσι ο συνολικός αριθμός των διεργασιών που ζητάει ο χρήστης θα συμπεριλαμβάνει και την γονική διεργασία.
-

Βεβαιωθείτε πως ο πολυδιεργασιακός κώδικας τρέχει σωστά και ο χρόνος εκτέλεσης μειώνεται όταν χρησιμοποιούνται περισσότερες διεργασίες. Αυτό προϋποθέτει πως το υπολογιστικό σύστημα θα είναι αδρανές, άρα θα τρέχει μόνο το δικό σας εκτελέσιμο, και πως ο αριθμός των διεργασιών-εργατών δεν είναι μεγαλύτερος από τον αριθμό των διαθέσιμων επεξεργαστικών πυρήνων.